NO GLYPH Tauri + React + Rust — Calculator App (হাতে-ধরা গাইড)

Calculator App

> **লক্ষ্য:** এই প্রজেক্ট করলে তুমি Tauri + Rust এর মূল ধারণা বুঝবে এবং পরবর্তী যেকোনো ডেস্কটপ অ্যাপ একই টেমপ্লেটে বানাতে পারবে।

ধাপ ১ — Prerequisites (আগে যা লাগবে)

VS Code খোলার আগে এগুলো ইনস্টল করো:

১.১ Node.js ইনস্টল করো

👉 https://nodejs.org থেকে **LTS** ভার্সন ডাউনলোড করো।

চেক করো:

```
node -v     # v18+ দেখালে ঠিক আছে
npm -v      # 9+ দেখালে ঠিক আছে
```

১.২ Rust ইনস্টল করো

👉 https://rustup.rs — ওখানে যাও, command কপি করে terminal এ দাও।

Windows এ:

```
# rustup-init.exe ডাউনলোড করে চালাও
# তারপর চেক:
rustc --version   # rustc 1.7x.x দেখালে ঠিক আছে
cargo --version   # cargo 1.7x.x দেখালে ঠিক আছে
```

১.৩ Tauri এর System Dependencies

Windows এ:

- Microsoft Visual Studio C++ Build Tools লাগবে

- WebView2 (Windows 11 এ আগে থেকেই থাকে)
- 👉 https://tauri.app/start/prerequisites/ থেকে দেখো

একটা test:

```
cargo install tauri-cli
```

এটা শেষ হলে তুমি রেডি।

ধাপ ২ — প্রজেক্ট তৈরি করো

তোমার command গুলো এভাবে চালাও:

```
# ⚠ তুমি vanilla-ts দিয়েছিলে — কিন্তু React এর জন্য react-ts লাগবে
npx create-tauri-app tauri-calculator -m npm -t react-ts -y
cd tauri-calculator
npm install
npm run tauri dev
```

> **Note:** `vanilla-ts` → plain TypeScript দেয়, `react-ts` → React + TypeScript দেয়। তোমার বাকি সব কমান্ড একই থাকবে, শুধু template বদলালাম।

`npm run tauri dev` চালালে:

- একটা window খুলবে (এটাই তোমার desktop app!)
- Code সেভ করলে auto-reload হবে

ধাপ ৩ — ফাইল স্ট্রাকচার বোঝো

```
tauri-calculator/
│
├── src/                          ← React (Frontend) এখানে
│   ├── components/               ← সব UI component (নিজে
বানাবে)
│   │   ├── Calculator.tsx        ← মূল calculator component
│   │   └── Display.tsx           ← display অংশ
│   ├── App.tsx                   ← Root component
│   ├── App.css                   ← App এর styles
```

```
│   ├── main.tsx                    ← React entry point (হাত দেবে না)
│   └── index.css                   ← Global styles
│
├── src-tauri/                      ← Rust (Backend) এখানে
│   ├── src/
│   │   ├── main.rs                 ← App চালু হয় এখান থেকে
│   │   └── lib.rs                  ← তোমার Rust commands এখানে লিখবে
│   ├── Cargo.toml                  ← Rust এর package.json (dependencies)
│   ├── tauri.conf.json             ← App এর config (নাম, icon, window size)
│   └── icons/                     ← App icon গুলো
│
├── public/                         ← Static files (images, fonts)
├── index.html                      ← HTML template (হাত দেবে না)
├── package.json                    ← Node dependencies
├── tsconfig.json                   ← TypeScript config
└── vite.config.ts                  ← Build tool config
```

🧠 মনে রাখো:

ফোল্ডার	ভাষা	কাজ
`src/`	React + TypeScript	যা দেখা যায় (UI)
`src-tauri/src/`	Rust	যা দেখা যায় না (logic, system access)
দুইটা IPC দিয়ে কথা বলে	`invoke()`	Frontend → Backend

ধাপ ৪ — Rust Backend লেখো

৪.১ `src-tauri/src/lib.rs` ফাইল খোলো

এই ফাইলে তোমার সব Rust "commands" থাকবে। এই commands গুলোই React থেকে ডাকা যাবে।

পুরো ফাইলটা এভাবে লেখো:

```
// lib.rs — এখানে সব Tauri commands থাকবে

// #[tauri::command] মানে এই function টা React থেকে ডাকা যাবে
#[tauri::command]
fn calculate(a: f64, operator: String, b: f64) -> Result<f6
4, String> {
    // match মানে switch-case এর মতো
    match operator.as_str() {
        "+" => Ok(a + b),
        "-" => Ok(a - b),
        "*" => Ok(a * b),
        "/" => {
            if b == 0.0 {
                // Error পাঠাও — React এ catch হবে
                Err(String::from("শূন্য দিয়ে ভাগ করা যায় না!"))
            } else {
                Ok(a / b)
            }
        }
        _ => Err(format!("অজানা অপারেটর: {}", operator)),
    }
}

// App চালু হলে এই function run হয়
#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_opener::init())
        // নতুন command বানালে এখানে যোগ করতে হবে
        .invoke_handler(tauri::generate_handler![calculat
e])
        .run(tauri::generate_context!())
        .expect("Tauri app চালু করতে সমস্যা হয়েছে");
}
```

৪.২ Rust এর মূল ধারণা (ছোট করে)

```
// f64 মানে decimal number (64-bit float)
// String মানে text
// Result<f64, String> মানে: হয় f64 দেবে, নাহয় String error
দেবে
// Ok(value) মানে সফল হয়েছে
// Err(msg) মানে ব্যর্থ হয়েছে
// match → switch-case এর মতো, কিন্তু আরও powerful
```

ধাপ ৫ — React Frontend লেখো

৫.১ `src/components/` ফোল্ডার বানাও

VS Code terminal এ:

```
mkdir src/components
```

৫.২ `src/components/Display.tsx` বানাও

```
// Display.tsx — Calculator এর screen অংশ

interface DisplayProps {
  value: string;      // দেখানোর জন্য value
  expression: string; // যেমন "12 + 5 ="
}

export default function Display({ value, expression }: Disp
layProps) {
  return (
    <div className="display">
      <div className="expression">{expression}</div>
      <div className="value">{value}</div>
    </div>
  );
}
```

৫.৩ `src/components/Calculator.tsx` বানাও

এটাই মূল component। এখানে Rust এর সাথে কথা বলা হয়।

```
// Calculator.tsx

import { useState } from "react";
import { invoke } from "@tauri-apps/api/core"; // ← এই দিয়ে
Rust ডাকি
import Display from "./Display";

export default function Calculator() {
  // useState মানে এই values গুলো বদলালে screen আপডেট হবে
  const [display, setDisplay] = useState("0");         // s
creen এ যা দেখায়
  const [firstNum, setFirstNum] = useState("");         //
প্রথম সংখ্যা
  const [operator, setOperator] = useState("");         //
অপারেটর (+, -, *, /)
  const [expression, setExpression] = useState("");     //
উপরে যা দেখায়
  const [waitingForSecond, setWaitingForSecond] = useState
(false); // অপারেটর চাপার পর

  // সংখ্যার বোতাম চাপলে
  function handleNumber(num: string) {
    if (waitingForSecond) {
      setDisplay(num);
      setWaitingForSecond(false);
    } else {
      setDisplay(display === "0" ? num : display + num);
    }
  }

  // দশমিক বোতাম চাপলে
  function handleDecimal() {
    if (!display.includes(".")) {
      setDisplay(display + ".");
    }
  }
```

```
// অপারেটর বোতাম চাপলে (+, -, *, /)
function handleOperator(op: string) {
  setFirstNum(display);
  setOperator(op);
  setExpression(`${display} ${op}`);
  setWaitingForSecond(true);
}

// = বোতাম চাপলে Rust কে ডাকো
async function handleEquals() {
  if (!firstNum || !operator) return;

  setExpression(`${firstNum} ${operator} ${display} =`);

  try {
    // ← এখানেই Rust function ডাকা হচ্ছে!
    // invoke("rust_function_name", { arguments })
    const result = await invoke<number>("calculate", {
      a: parseFloat(firstNum),
      operator: operator,
      b: parseFloat(display),
    });

    // Rust থেকে result আসলে দেখাও
    setDisplay(String(result));
    setFirstNum("");
    setOperator("");
    setWaitingForSecond(true);
  } catch (error) {
    // Rust Err() পাঠালে এখানে আসবে
    setDisplay(String(error));
  }
}

// C (Clear) বোতাম চাপলে
function handleClear() {
  setDisplay("0");
```

```
  setFirstNum("");
  setOperator("");
  setExpression("");
  setWaitingForSecond(false);
}

// +/- বোতাম
function handleToggleSign() {
  setDisplay(String(parseFloat(display) * -1));
}

// % বোতাম
function handlePercent() {
  setDisplay(String(parseFloat(display) / 100));
}

// বোতামের layout (iPhone calculator এর মতো)
const buttons = [
  ["C", "+/-", "%", "÷"],
  ["7", "8", "9", "×"],
  ["4", "5", "6", "−"],
  ["1", "2", "3", "+"],
  ["0", ".", "="],
];

// বোতাম চাপলে কী করবে
function handleButton(btn: string) {
  if ("0123456789".includes(btn)) handleNumber(btn);
  else if (btn === ".") handleDecimal();
  else if (btn === "C") handleClear();
  else if (btn === "+/-") handleToggleSign();
  else if (btn === "%") handlePercent();
  else if (btn === "=") handleEquals();
  else if (btn === "÷") handleOperator("/");
  else if (btn === "×") handleOperator("*");
  else if (btn === "−") handleOperator("-");
  else if (btn === "+") handleOperator("+");
}
```

```
  // বোতামের ধরন (রঙের জন্য)
  function getButtonClass(btn: string) {
    if (btn === "C" || btn === "+/-" || btn === "%") return
"btn btn-gray";
    if (["÷", "×", "−", "+", "="].includes(btn)) return "bt
n btn-orange";
    if (btn === "0") return "btn btn-dark btn-wide";
    return "btn btn-dark";
  }

  return (
    <div className="calculator">
      <Display value={display} expression={expression} />

      <div className="buttons">
        {buttons.map((row, rowIndex) => (
          <div key={rowIndex} className="row">
            {row.map((btn) => (
              <button
                key={btn}
                className={getButtonClass(btn)}
                onClick={() => handleButton(btn)}
              >
                {btn}
              </button>
            ))}
          </div>
        ))}
      </div>
    </div>
  );
}
```

৫.৪ `src/App.tsx` পরিবর্তন করো

```
// App.tsx — Root component, Calculator কে এখানে রাখি
```

```
import Calculator from "./components/Calculator";
import "./App.css";

function App() {
  return (
    <div className="app">
      <Calculator />
    </div>
  );
}

export default App;
```

ধাপ ৬ — CSS Styling

৬.১ `src/App.css` পুরোটা মুছে এটা লেখো

```
:root {
  font-family: "Inter", system-ui, -apple-system, BlinkMacS
ystemFont, sans-serif;
  font-size: 16px;
  line-height: 1.6;
  font-weight: 400;

  color: #1a1a1a;
  background-color: #f5f7fa;

  --primary: #646cff;
  --primary-hover: #535bf2;
  --accent: #24c8db;

  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
}

/* Layout */
.container {
```

```
  margin: 0;
  padding-top: 12vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 1.5rem;
  text-align: center;
}

.row {
  display: flex;
  justify-content: center;
  gap: 1rem;
  flex-wrap: wrap;
}

/* Logo */
.logo {
  height: 6em;
  padding: 1.2em;
  transition: transform 0.3s ease, filter 0.4s ease;
}

.logo:hover {
  transform: scale(1.08);
}

.logo.vite:hover {
  filter: drop-shadow(0 0 1.8em #747bff);
}

.logo.react:hover {
  filter: drop-shadow(0 0 1.8em #61dafb);
}

.logo.tauri:hover {
  filter: drop-shadow(0 0 1.8em #24c8db);
}
```

```
/* Typography */
h1 {
  font-size: 2.2rem;
  font-weight: 600;
  margin-bottom: 0.5rem;
}

a {
  font-weight: 500;
  color: var(--primary);
  text-decoration: none;
  transition: color 0.25s ease;
}

a:hover {
  color: var(--primary-hover);
}

/* Inputs & Buttons */
input,
button {
  border-radius: 10px;
  border: 1px solid transparent;
  padding: 0.65em 1.2em;
  font-size: 1rem;
  font-family: inherit;

  color: #1a1a1a;
  background-color: #ffffff;

  transition: all 0.25s ease;
  box-shadow: 0 3px 6px rgba(0, 0, 0, 0.08);
}

input:focus,
button:focus {
  border-color: var(--primary);
```

```
  box-shadow: 0 0 0 3px rgba(100, 108, 255, 0.2);
}

/* Button */
button {
  cursor: pointer;
  font-weight: 500;
}

button:hover {
  border-color: var(--primary);
  background-color: #f0f3ff;
}

button:active {
  transform: scale(0.97);
  background-color: #e6e9ff;
}

/* Input spacing */
#greet-input {
  margin-right: 8px;
}

/* Dark Mode */
@media (prefers-color-scheme: dark) {
  :root {
    color: #f5f5f5;
    background-color: #1e1e1e;
  }

  a:hover {
    color: var(--accent);
  }

  input,
  button {
    color: #ffffff;
```

```
    background-color: rgba(255, 255, 255, 0.05);
    border-color: rgba(255, 255, 255, 0.08);
  }

  input:focus,
  button:focus {
    box-shadow: 0 0 0 3px rgba(36, 200, 219, 0.25);
  }

  button:hover {
    background-color: rgba(255, 255, 255, 0.08);
  }

  button:active {
    background-color: rgba(255, 255, 255, 0.12);
  }
}
```

ধাপ ৭ — App চালাও

```
npm run tauri dev
```

Calculator window খুলবে! বোতাম চাপো, দেখো কাজ করছে কিনা।

ধাপ ৮ — Build করো (Final .exe বানাও)

```
npm run tauri build
```

শেষ হলে:

- Windows: `src-tauri/target/release/tauri-calculator.exe`
- Setup installer: `src-tauri/target/release/bundle/`

ধাপ ৯ — App এর নাম ও Window Size বদলাও

`src-tauri/tauri.conf.json` খোলো:

```
{
  "app": {
    "windows": [
      {
        "title": "আমার Calculator",
        "width": 360,
        "height": 620,
        "resizable": false
      }
    ]
  },
  "bundle": {
    "identifier": "com.yourname.calculator",
    "productName": "Calculator"
  }
}
```

🧠 Key Concepts — মনে রাখো

Tauri এর মূল কথা:

```
React (দেখায়) ←→ Tauri Bridge ←→ Rust (কাজ করে)
```

নতুন Rust Command বানাতে হলে:

lib.rs এ:

```
#[tauri::command]
fn my_new_command(input: String) -> String {
    format!("তুমি পাঠিয়েছ: {}", input)
}

// এবং invoke_handler এ যোগ করো:
.invoke_handler(tauri::generate_handler![calculate, my_new_
command])
```

React এ:

```
const result = await invoke<string>("my_new_command", { inp
ut: "hello" });
```

Rust এর ৩টা জিনিস যা বারবার দেখবে:

Syntax	মানে
`fn name(param: Type) -> ReturnType`	function definition
`Result<Ok, Err>`	হয় সফল, নাহয় error
`match value { pattern => action }`	switch-case

পরের প্রজেক্টের টেমপ্লেট

একই structure ব্যবহার করে যেকোনো app বানাতে পারবে:

```
নতুন project:
1. npx create-tauri-app my-app -m npm -t react-ts -y
2. src/components/ → তোমার UI
3. src-tauri/src/lib.rs → তোমার Rust logic
4. invoke() দিয়ে দুইটা যুক্ত করো
```

আইডিয়া:

- File reader app → Rust file system access
- System info app → Rust CPU/RAM data
- Note taking app → Rust SQLite database
- Screenshot tool → Rust screen capture

সমস্যা হলে

সমস্যা	সমাধান
`cargo not found`	Rust ইনস্টল হয়নি, rustup.rs যাও
`WebView2 error`	Windows এ WebView2 ইনস্টল করো
`invoke failed`	lib.rs এ `generate_handler!` এ command আছে কিনা দেখো
build error in Rust	terminal এর error message পড়ো, line number দেখাবে

এই গাইড দিয়ে তুমি Tauri এর সব মূল concept বুঝতে পারবে। Happy coding! 🚀